

# EC4219: Software Engineering

## Lecture 8 — Program Verification (3) *Inductive Assertion Method (Cont'd)*

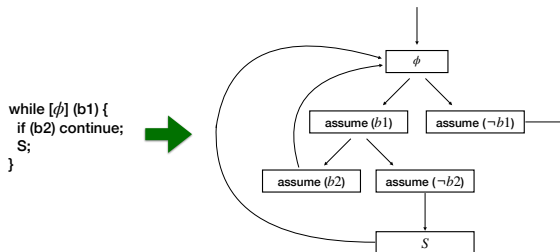
Sunbeom So  
2025 Spring

# This Lecture: Predicate Transformer in a Different Style

- Directly encoding the effects of `continue` and `break` is hard, because we should consider execution flows outside the given statement.

$$\begin{aligned}\text{pre}(F, \text{continue}) &\iff ? \\ \text{pre}(F, \text{break}) &\iff ?\end{aligned}$$

- To analyze programs with `continue` and `break`, we construct control-flow graphs and generate VCs over the basic paths in them.



- If a program does not contain them, we can verify it without control-flow graphs!

# Tiny Imperative Language

To illustrate the approach, we will consider a very small imperative language.

$$\begin{aligned} lv &\rightarrow x \mid x[e] \\ e &\rightarrow n \mid lv \mid e_1 \oplus e_2 \mid \text{true} \mid \text{false} \\ &\quad \mid e_1 \prec e_2 \mid \neg e \mid e_1 \&\& e_2 \mid e_1 \parallel e_2 \\ c &\rightarrow lv := e \mid c_1; c_2 \mid \text{if } e \text{ } c_1 \text{ } c_2 \mid \text{while } [\phi] \text{ } e \text{ } c \end{aligned}$$

- $\oplus$  and  $\prec$  are standard arithmetic ( $+$ ,  $-$ ,  $\dots$ ) and comparison ( $>$ ,  $\geq$ ,  $\dots$ ) operators, respectively.
- $\phi$  in the while-loop is a candidate invariant expressed in a FOL formula.

$$\mathbf{pre}(F, \text{if } e \text{ } c_1 \text{ } c_2) \iff e \rightarrow \mathbf{pre}(F, c_1) \wedge \neg e \rightarrow \mathbf{pre}(F, c_2)$$

- If  $e$  holds,  $F$  must hold after executing the if-branch ( $c_1$ ).
- Otherwise,  $F$  must hold after exeucintg the else-branch ( $c_2$ ).

## Example: If-Statement

Consider the following statement  $S$

$$S : x := y+1; \text{ if } x > 0 \text{ then } z := 1 \text{ else } z := -1$$

and its Hoare triple

$$\{y > -1\} S \{z = 1\}.$$

Determine the validity of the Hoare triple.

## Example: If-Statement

Consider the following statement  $S$

$S : x := y+1; \text{ if } x > 0 \text{ then } z := 1 \text{ else } z := -1$

and its Hoare triple

$$\{y > -1\} S \{z = 1\}.$$

Determine the validity of the Hoare triple.

$$\begin{aligned} & \{y > -1\} S \{z = 1\} \\ \iff & y > -1 \rightarrow \text{pre}(z = 1, S) \\ \iff & y > -1 \rightarrow \text{pre}(x > 0 \rightarrow \text{pre}(z = 1, z := 1) \\ & \quad \wedge \neg(x > 0) \rightarrow \text{pre}(z = 1, z := -1), x := y + 1) \\ \iff & y > -1 \rightarrow \text{pre}(x > 0 \rightarrow 1 = 1 \wedge \neg(x > 0) \rightarrow -1 = 1, x := y + 1) \\ \iff & y > -1 \rightarrow (y + 1 > 0 \rightarrow 1 = 1 \wedge \neg(y + 1 > 0) \rightarrow -1 = 1) \end{aligned}$$

The VC is valid.

# Loops

- Unfortunately, we cannot exactly compute the weakest preconditions for loops.
- We just rely on the annotated invariant  $\phi$ .

$$\mathbf{pre}(F, \text{while } [\phi] \text{ e } c) \iff \phi$$

Idea: since the invariant  $\phi$  holds before entering the loop (by definition), we just compute the precondition as  $\phi$ .

- We should impose extra conditions (why?).

- Unfortunately, we cannot exactly compute the weakest preconditions for loops.
- We just rely on the annotated invariant  $\phi$ .

$$\text{pre}(F, \text{while } [\phi] \ e \ c) \iff \phi$$

Idea: since the invariant  $\phi$  holds before entering the loop (by definition), we just compute the precondition as  $\phi$ .

- We should impose extra conditions (why?).
  - ① We haven't checked that  $\phi$  is a true **inductive loop invariant**.
  - ② We haven't checked that  $\phi \wedge \neg e$  is sufficient to establish  $F$  (i.e., we haven't check the validity of  $\phi \wedge \neg e \rightarrow F$ ).



# Invariant vs. Inductive Invariant

- Suppose  $\phi$  is an annotated loop invariant for `while [ $\phi$ ] e c`.
- We should check  $\{\phi \wedge e\} c \{\phi\}$  is valid or not.
- Even if  $\phi$  is true during the loop, the Hoare triple may not be valid.
- For example, consider  $\phi : j \geq 1$  and the code

`i := 1; j := 1; while i < n do j := j + i; i := i + 1`

$\phi$  is not an inductive, provable invariant (even if  $\phi$  holds!), since the Hoare triple is invalid:

$$\{j \geq 1 \wedge i < n\} j := j + i; i := i + 1 \{j \geq 1\}$$

However, the strengthened invariant  $j \geq 1 \wedge i \geq 1$  is an inductive invariant.

- In verification, we are typically interested in finding inductive invariants.

## Extra Conditions for Inductive Loop Invariants

$$\begin{aligned} VC(F, \text{while } [\phi] \ e \ c) \\ \iff (\phi \wedge e \rightarrow \mathbf{pre}(\phi, c)) \wedge (\phi \wedge \neg e \rightarrow F) \wedge VC(\phi, c) \end{aligned}$$

The other cases should be defined as well, because we may have nested loops and corresponding invariants.

$$\begin{aligned} VC(F, lv := e) &\iff \text{true} \\ VC(F, c_1; c_2) &\iff VC(F, c_2) \wedge VC(\mathbf{pre}(F, c_2), c_1) \\ VC(F, \text{if } e \ c_1 \ c_2) &\iff VC(F, c_1) \wedge VC(F, c_2) \end{aligned}$$

# Final: Verification of Hoare Triple

To show the validity of  $\{P\} S \{Q\}$ , we should check the validity of the following formula.

$$VC(Q, S) \wedge (P \rightarrow \mathbf{pre}(Q, S))$$

That is, if the formula is valid, we prove the partial correctness of the implementation.

# Summary

- A different style of the predicate transformer without control-flow graphs.
  - ▶ In the previous version, the inductiveness is checked in each basic path.
  - ▶ In the new version, we explicitly impose extra conditions for it.
- Invariant vs. Inductive Invariant